

Since we often want our RDDs in the reverse order, the `sortByKey()` function takes a parameter called `ascending` indicating whether we want it in ascending order (it defaults to `true`). Sometimes we want a different sort order entirely, and to support this we can provide our own comparison function. In Examples 4-19 through 4-21, we will sort our RDD by converting the integers to strings and using the string comparison functions.

Example 4-19. Custom sort order in Python, sorting integers as if strings

```
rdd.sortByKey(ascending=True, numPartitions=None, keyfunc = lambda x: str(x))
```

Example 4-20. Custom sort order in Scala, sorting integers as if strings

```
val input: RDD[(Int, Venue)] = ...
implicit val sortIntegersByString = new Ordering[Int] {
  override def compare(a: Int, b: Int) = a.toString.compare(b.toString)
}
rdd.sortByKey()
```

Example 4-21. Custom sort order in Java, sorting integers as if strings

```
class IntegerComparator implements Comparator<Integer> {
  public int compare(Integer a, Integer b) {
    return String.valueOf(a).compareTo(String.valueOf(b))
  }
}
rdd.sortByKey(comp)
```

Actions Available on Pair RDDs

As with the transformations, all of the traditional actions available on the base RDD are also available on pair RDDs. Some additional actions are available on pair RDDs to take advantage of the key/value nature of the data; these are listed in Table 4-3.

Table 4-3. Actions on pair RDDs (example $\{(1, 2), (3, 4), (3, 6)\}$)

Function	Description	Example	Result
<code>countByKey()</code>	Count the number of elements for each key.	<code>rdd.countByKey()</code>	$\{(1, 1), (3, 2)\}$
<code>collectAsMap()</code>	Collect the result as a map to provide easy lookup.	<code>rdd.collectAsMap()</code>	$\text{Map}\{(1, 2), (3, 4), (3, 6)\}$
<code>lookup(key)</code>	Return all values associated with the provided key.	<code>rdd.lookup(3)</code>	$[4, 6]$

There are also multiple other actions on pair RDDs that save the RDD, which we will describe in [Chapter 5](#).

Data Partitioning (Advanced)

The final Spark feature we will discuss in this chapter is how to control datasets' partitioning across nodes. In a distributed program, communication is very expensive, so laying out data to minimize network traffic can greatly improve performance. Much like how a single-node program needs to choose the right data structure for a collection of records, Spark programs can choose to control their RDDs' partitioning to reduce communication. Partitioning will not be helpful in all applications—for example, if a given RDD is scanned only once, there is no point in partitioning it in advance. It is useful only when a dataset is reused *multiple times* in key-oriented operations such as joins. We will give some examples shortly.

Spark's partitioning is available on all RDDs of key/value pairs, and causes the system to group elements based on a function of each key. Although Spark does not give explicit control of which worker node each key goes to (partly because the system is designed to work even if specific nodes fail), it lets the program ensure that a *set* of keys will appear together on *some* node. For example, you might choose to hash-partition an RDD into 100 partitions so that keys that have the same hash value modulo 100 appear on the same node. Or you might range-partition the RDD into sorted ranges of keys so that elements with keys in the same range appear on the same node.

As a simple example, consider an application that keeps a large table of user information in memory—say, an RDD of (UserID, UserInfo) pairs, where UserInfo contains a list of topics the user is subscribed to. The application periodically combines this table with a smaller file representing events that happened in the past five minutes—say, a table of (UserID, LinkInfo) pairs for users who have clicked a link on a website in those five minutes. For example, we may wish to count how many users visited a link that was *not* to one of their subscribed topics. We can perform this combination with Spark's `join()` operation, which can be used to group the User Info and LinkInfo pairs for each UserID by key. Our application would look like

Example 4-22.

Example 4-22. Scala simple application

```
// Initialization code; we load the user info from a Hadoop SequenceFile on HDFS.
// This distributes elements of userData by the HDFS block where they are found,
// and doesn't provide Spark with any way of knowing in which partition a
// particular UserID is located.
val sc = new SparkContext(...)
val userData = sc.sequenceFile[UserID, UserInfo]("hdfs://...").persist()

// Function called periodically to process a logfile of events in the past 5 minutes;
```

```
// we assume that this is a SequenceFile containing (UserID, LinkInfo) pairs.
def processNewLogs(logFileName: String) {
  val events = sc.sequenceFile[UserID, LinkInfo](logFileName)
  val joined = userData.join(events) // RDD of (UserID, (UserInfo, LinkInfo)) pairs
  val offTopicVisits = joined.filter {
    case (userId, (userInfo, linkInfo)) => // Expand the tuple into its components
      !userInfo.topics.contains(linkInfo.topic)
  }.count()
  println("Number of visits to non-subscribed topics: " + offTopicVisits)
}
```

This code will run fine as is, but it will be inefficient. This is because the `join()` operation, called each time `processNewLogs()` is invoked, does not know anything about how the keys are partitioned in the datasets. By default, this operation will hash all the keys of *both* datasets, sending elements with the same key hash across the network to the same machine, and then join together the elements with the same key on that machine (see [Figure 4-4](#)). Because we expect the `userData` table to be much larger than the small log of events seen every five minutes, this wastes a lot of work: the `userData` table is hashed and shuffled across the network on every call, even though it doesn't change.

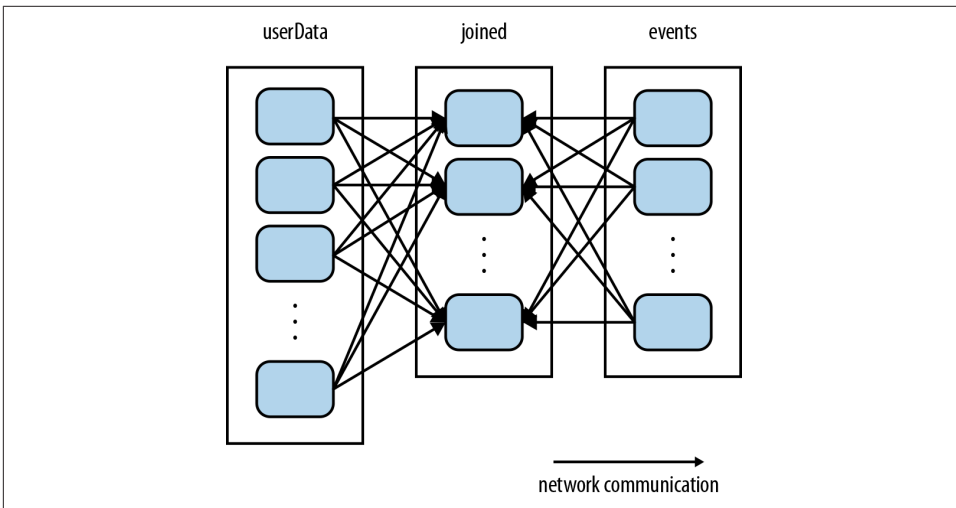


Figure 4-4. Each join of `userData` and `events` without using `partitionBy()`

Fixing this is simple: just use the `partitionBy()` transformation on `userData` to hash-partition it at the start of the program. We do this by passing a `spark.HashPartitioner` object to `partitionBy`, as shown in [Example 4-23](#).

Example 4-23. Scala custom partitioner

```
val sc = new SparkContext(...)
val userData = sc.sequenceFile[UserID, UserInfo]("hdfs://...")
    .partitionBy(new HashPartitioner(100)) // Create 100 partitions
    .persist()
```

The `processNewLogs()` method can remain unchanged: the events RDD is local to `processNewLogs()`, and is used only once within this method, so there is no advantage in specifying a partitioner for events. Because we called `partitionBy()` when building `userData`, Spark will now know that it is hash-partitioned, and calls to `join()` on it will take advantage of this information. In particular, when we call `userData.join(events)`, Spark will shuffle only the events RDD, sending events with each particular `UserID` to the machine that contains the corresponding hash partition of `userData` (see [Figure 4-5](#)). The result is that a lot less data is communicated over the network, and the program runs significantly faster.

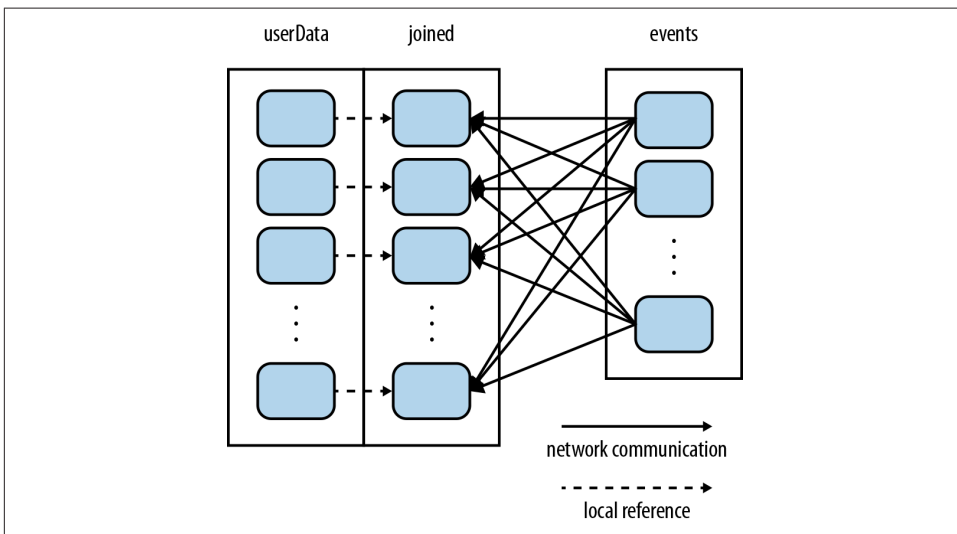


Figure 4-5. Each join of `userData` and `events` using `partitionBy()`

Note that `partitionBy()` is a transformation, so it always returns a *new* RDD—it does not change the original RDD in place. RDDs can never be modified once created. Therefore it is important to persist and save as `userData` the result of `partitionBy()`, not the original `sequenceFile()`. Also, the 100 passed to `partitionBy()` represents the number of partitions, which will control how many parallel tasks perform further operations on the RDD (e.g., joins); in general, make this at least as large as the number of cores in your cluster.



Failure to persist an RDD after it has been transformed with `partitionBy()` will cause subsequent uses of the RDD to repeat the partitioning of the data. Without persistence, use of the partitioned RDD will cause reevaluation of the RDDs complete lineage. That would negate the advantage of `partitionBy()`, resulting in repeated partitioning and shuffling of data across the network, similar to what occurs without any specified partitioner.

In fact, many other Spark operations automatically result in an RDD with known partitioning information, and many operations other than `join()` will take advantage of this information. For example, `sortByKey()` and `groupByKey()` will result in range-partitioned and hash-partitioned RDDs, respectively. On the other hand, operations like `map()` cause the new RDD to *forget* the parent's partitioning information, because such operations could theoretically modify the key of each record. The next few sections describe how to determine how an RDD is partitioned, and exactly how partitioning affects the various Spark operations.



Partitioning in Java and Python

Spark's Java and Python APIs benefit from partitioning in the same way as the Scala API. However, in Python, you cannot pass a Hash Partitioner object to `partitionBy`; instead, you just pass the number of partitions desired (e.g., `rdd.partitionBy(100)`).

Determining an RDD's Partitioner

In Scala and Java, you can determine how an RDD is partitioned using its `partitioner` property (or `partitioner()` method in Java).² This returns a `scala.Option` object, which is a Scala class for a container that may or may not contain one item. You can call `isDefined()` on the `Option` to check whether it has a value, and `get()` to get this value. If present, the value will be a `spark.Partitioner` object. This is essentially a function telling the RDD which partition each key goes into; we'll talk more about this later.

The `partitioner` property is a great way to test in the Spark shell how different Spark operations affect partitioning, and to check that the operations you want to do in your program will yield the right result (see [Example 4-24](#)).

² The Python API does not yet offer a way to query partitioners, though it still uses them internally.

Example 4-24. Determining partitioner of an RDD

```
scala> val pairs = sc.parallelize(List((1, 1), (2, 2), (3, 3)))
pairs: spark.RDD[(Int, Int)] = ParallelCollectionRDD[0] at parallelize at <console>:12

scala> pairs.partitioner
res0: Option[spark.Partitioner] = None

scala> val partitioned = pairs.partitionBy(new spark.HashPartitioner(2))
partitioned: spark.RDD[(Int, Int)] = ShuffledRDD[1] at partitionBy at <console>:14

scala> partitioned.partitioner
res1: Option[spark.Partitioner] = Some(spark.HashPartitioner@5147788d)
```

In this short session, we created an RDD of (Int, Int) pairs, which initially have no partitioning information (an Option with value None). We then created a second RDD by hash-partitioning the first. If we actually wanted to use partitioned in further operations, then we should have appended persist() to the third line of input, in which partitioned is defined. This is for the same reason that we needed persist() for userData in the previous example: without persist(), subsequent RDD actions will evaluate the entire lineage of partitioned, which will cause pairs to be hash-partitioned over and over.

Operations That Benefit from Partitioning

Many of Spark's operations involve shuffling data by key across the network. All of these will benefit from partitioning. As of Spark 1.0, the operations that benefit from partitioning are cogroup(), groupWith(), join(), leftOuterJoin(), rightOuterJoin(), groupByKey(), reduceByKey(), combineByKey(), and lookup().

For operations that act on a single RDD, such as reduceByKey(), running on a pre-partitioned RDD will cause all the values for each key to be computed *locally* on a single machine, requiring only the final, locally reduced value to be sent from each worker node back to the master. For binary operations, such as cogroup() and join(), pre-partitioning will cause at least one of the RDDs (the one with the known partitioner) to not be shuffled. If both RDDs have the *same* partitioner, and if they are cached on the same machines (e.g., one was created using mapValues() on the other, which preserves keys and partitioning) or if one of them has not yet been computed, then no shuffling across the network will occur.

Operations That Affect Partitioning

Spark knows internally how each of its operations affects partitioning, and automatically sets the partitioner on RDDs created by operations that partition the data. For example, suppose you called join() to join two RDDs; because the elements with the same key have been hashed to the same machine, Spark knows that the result is

hash-partitioned, and operations like `reduceByKey()` on the join result are going to be significantly faster.

The flipside, however, is that for transformations that *cannot* be guaranteed to produce a known partitioning, the output RDD will not have a `partitioner` set. For example, if you call `map()` on a hash-partitioned RDD of key/value pairs, the function passed to `map()` can in theory change the key of each element, so the result will not have a `partitioner`. Spark does not analyze your functions to check whether they retain the key. Instead, it provides two other operations, `mapValues()` and `flatMapValues()`, which guarantee that each tuple's key remains the same.

All that said, here are all the operations that result in a `partitioner` being set on the output RDD: `cogroup()`, `groupWith()`, `join()`, `leftOuterJoin()`, `rightOuterJoin()`, `groupByKey()`, `reduceByKey()`, `combineByKey()`, `partitionBy()`, `sort()`, `mapValues()` (if the parent RDD has a `partitioner`), `flatMapValues()` (if parent has a `partitioner`), and `filter()` (if parent has a `partitioner`). All *other* operations will produce a result with no `partitioner`.

Finally, for binary operations, *which* `partitioner` is set on the output depends on the parent RDDs' `partitioners`. By default, it is a hash `partitioner`, with the number of partitions set to the level of parallelism of the operation. However, if one of the parents has a `partitioner` set, it will be that `partitioner`; and if both parents have a `partitioner` set, it will be the `partitioner` of the first parent.

Example: PageRank

As an example of a more involved algorithm that can benefit from RDD partitioning, we consider PageRank. The PageRank algorithm, named after Google's Larry Page, aims to assign a measure of importance (a "rank") to each document in a set based on how many documents have links to it. It can be used to rank web pages, of course, but also scientific articles, or influential users in a social network.

PageRank is an iterative algorithm that performs many joins, so it is a good use case for RDD partitioning. The algorithm maintains two datasets: one of `(pageID, link List)` elements containing the list of neighbors of each page, and one of `(pageID, rank)` elements containing the current rank for each page. It proceeds as follows:

1. Initialize each page's rank to 1.0.
2. On each iteration, have page `p` send a contribution of $\text{rank}(p) / \text{numNeighbors}(p)$ to its neighbors (the pages it has links to).
3. Set each page's rank to $0.15 + 0.85 * \text{contributionsReceived}$.

The last two steps repeat for several iterations, during which the algorithm will converge to the correct PageRank value for each page. In practice, it's typical to run about 10 iterations.

Example 4-25 gives the code to implement PageRank in Spark.

Example 4-25. Scala PageRank

```
// Assume that our neighbor list was saved as a Spark objectFile
val links = sc.objectFile[(String, Seq[String])]("links")
               .partitionBy(new HashPartitioner(100))
               .persist()

// Initialize each page's rank to 1.0; since we use mapValues, the resulting RDD
// will have the same partitioner as links
var ranks = links.mapValues(v => 1.0)

// Run 10 iterations of PageRank
for (i <- 0 until 10) {
  val contributions = links.join(ranks).flatMap {
    case (pageId, (links, rank)) =>
      links.map(dest => (dest, rank / links.size))
  }
  ranks = contributions.reduceByKey((x, y) => x + y).mapValues(v => 0.15 + 0.85*v)
}

// Write out the final ranks
ranks.saveAsTextFile("ranks")
```

That's it! The algorithm starts with a `ranks` RDD initialized at 1.0 for each element, and keeps updating the `ranks` variable on each iteration. The body of `PageRank` is pretty simple to express in Spark: it first does a `join()` between the current `ranks` RDD and the static `links` one, in order to obtain the link list and rank for each page ID together, then uses this in a `flatMap` to create “contribution” values to send to each of the page's neighbors. We then add up these values by page ID (i.e., by the page receiving the contribution) and set that page's rank to $0.15 + 0.85 * \text{contributionsReceived}$.

Although the code itself is simple, the example does several things to ensure that the RDDs are partitioned in an efficient way, and to minimize communication:

1. Notice that the `links` RDD is joined against `ranks` on each iteration. Since `links` is a static dataset, we partition it at the start with `partitionBy()`, so that it does not need to be shuffled across the network. In practice, the `links` RDD is also likely to be much larger in terms of bytes than `ranks`, since it contains a list of neighbors for each page ID instead of just a `Double`, so this optimization saves

considerable network traffic over a simple implementation of PageRank (e.g., in plain MapReduce).

2. For the same reason, we call `persist()` on links to keep it in RAM across iterations.
3. When we first create ranks, we use `mapValues()` instead of `map()` to preserve the partitioning of the parent RDD (links), so that our first join against it is cheap.
4. In the loop body, we follow our `reduceByKey()` with `mapValues()`; because the result of `reduceByKey()` is already hash-partitioned, this will make it more efficient to join the mapped result against links on the next iteration.



To maximize the potential for partitioning-related optimizations, you should use `mapValues()` or `flatMapValues()` whenever you are not changing an element's key.

Custom Partitioners

While Spark's `HashPartitioner` and `RangePartitioner` are well suited to many use cases, Spark also allows you to tune how an RDD is partitioned by providing a custom `Partitioner` object. This can help you further reduce communication by taking advantage of domain-specific knowledge.

For example, suppose we wanted to run the PageRank algorithm in the previous section on a set of web pages. Here each page's ID (the key in our RDD) will be its URL. Using a simple hash function to do the partitioning, pages with similar URLs (e.g., <http://www.cnn.com/WORLD> and <http://www.cnn.com/US>) might be hashed to completely different nodes. However, we know that web pages within the same domain tend to link to each other a lot. Because PageRank needs to send a message from each page to each of its neighbors on each iteration, it helps to group these pages into the same partition. We can do this with a custom `Partitioner` that looks at just the domain name instead of the whole URL.

To implement a custom partitioner, you need to subclass the `org.apache.spark.Partitioner` class and implement three methods:

- `numPartitions: Int`, which returns the number of partitions you will create.
- `getPartition(key: Any): Int`, which returns the partition ID (0 to `numPartitions-1`) for a given key.
- `equals()`, the standard Java equality method. This is important to implement because Spark will need to test your `Partitioner` object against other instances of itself when it decides whether two of your RDDs are partitioned the same way!

One gotcha is that if you rely on Java's `hashCode()` method in your algorithm, it can return negative numbers. You need to be careful to ensure that `getPartition()` always returns a nonnegative result.

Example 4-26 shows how we would write the domain-name-based partitioner sketched previously, which hashes only the domain name of each URL.

Example 4-26. Scala custom partitioner

```
class DomainNamePartitioner(numParts: Int) extends Partitioner {
  override def numPartitions: Int = numParts
  override def getPartition(key: Any): Int = {
    val domain = new Java.net.URL(key.toString).getHost()
    val code = (domain.hashCode % numPartitions)
    if (code < 0) {
      code + numPartitions // Make it non-negative
    } else {
      code
    }
  }
}
// Java equals method to let Spark compare our Partitioner objects
override def equals(other: Any): Boolean = other match {
  case dnp: DomainNamePartitioner =>
    dnp.numPartitions == numPartitions
  case _ =>
    false
}
```

Note that in the `equals()` method, we used Scala's pattern matching operator (`match`) to test whether `other` is a `DomainNamePartitioner`, and cast it if so; this is the same as using `instanceof()` in Java.

Using a custom `Partitioner` is easy: just pass it to the `partitionBy()` method. Many of the shuffle-based methods in Spark, such as `join()` and `groupByKey()`, can also take an optional `Partitioner` object to control the partitioning of the output.

Creating a custom Partitioner in Java is very similar to Scala: just extend the `spark.Partitioner` class and implement the required methods.

In Python, you do not extend a Partitioner class, but instead pass a hash function as an additional argument to `RDD.partitionBy()`. [Example 4-27](#) demonstrates.

Example 4-27. Python custom partitioner

```
import urlparse

def hash_domain(url):
    return hash(urlparse.urlparse(url).netloc)

rdd.partitionBy(20, hash_domain)  # Create 20 partitions
```

Note that the hash function you pass will be compared *by identity* to that of other RDDs. If you want to partition multiple RDDs with the same partitioner, pass the same function object (e.g., a global function) instead of creating a new `lambda` for each one!

Conclusion

In this chapter, we have seen how to work with key/value data using the specialized functions available in Spark. The techniques from [Chapter 3](#) also still work on our pair RDDs. In the next chapter, we will look at how to load and save data.